

# Guide de Déploiement GCP — Solarius Microservices

## Prérequis

1. **Compte Google Cloud** avec facturation activée
2. **gcloud CLI** installé ([installation](https://cloud.google.com/sdk/docs/install) (<https://cloud.google.com/sdk/docs/install>))
3. **Docker** installé localement
4. **Compte GitHub** avec le repository `solarius-microservices`

## Coût estimé (3 plateformes, usage modéré)

| Service                             | Coût mensuel estimé  |
|-------------------------------------|----------------------|
| Cloud Run (Julia) — scale-to-zero   | 15-40 €              |
| Cloud Run (Python) — scale-to-zero  | 10-30 €              |
| Artifact Registry (stockage images) | ~2 €                 |
| <b>Total</b>                        | <b>~27-72 €/mois</b> |

💡 **Scale-to-zero** : quand personne n'utilise les microservices, le coût tombe à ~2 €/mois (juste le stockage). Cloud Run facture uniquement le temps de calcul actif.

## Étape 1 : Configuration GCP initiale

### 1.1 Créer le projet

```
# Se connecter à GCP
gcloud auth login

# Créer le projet
gcloud projects create solarius-microservices --name="Solarius Microservices"
gcloud config set project solarius-microservices

# Activer la facturation (via la console GCP)
# → https://console.cloud.google.com/billing
```

### 1.2 Exécuter le script de configuration

```
cd scripts/
chmod +x setup-gcp.sh
./setup-gcp.sh
```

Ce script :

- Active les APIs nécessaires (Cloud Run, Artifact Registry, Secret Manager)

- Crée le dépôt Docker (Artifact Registry)
- Crée un Service Account pour le CI/CD GitHub Actions
- Génère une clé JSON pour l'authentification

## Étape 2 : Configuration GitHub

### 2.1 Créer le repository

```
cd solarius-microservices/
git init
git add .
git commit -m "Initial commit – Julia Geostat + Python Viz microservices"
git branch -M main
git remote add origin git@github.com:SolariusTechnology/solarius-microservices.git
git push -u origin main
```

### 2.2 Configurer les secrets GitHub

Allez dans **GitHub → Repository → Settings → Secrets and variables → Actions**

Créez ces secrets :

| Secret                   | Valeur                                                  |
|--------------------------|---------------------------------------------------------|
| GCP_SA_KEY               | Contenu complet du fichier <code>gcp-sa-key.json</code> |
| API_KEY_GEOECONOMIX      | Clé API pour GEO-ECONOMIX (générez un UUID)             |
| API_KEY_GEOMATRIX        | Clé API pour GeoMatrix                                  |
| API_KEY_TERRAEXPLORATION | Clé API pour TerraExploration                           |

💡 Pour générer des clés API sécurisées :

```
bash
python3 -c "import uuid; print(f'sk-solarius-{uuid.uuid4().hex}')"

```

## Étape 3 : Premier déploiement

Poussez sur `main` — le CI/CD se déclenche automatiquement :

```
git push origin main
```

Le workflow GitHub Actions va :

1. Builder les 2 images Docker
2. Les pousser sur Artifact Registry
3. Déployer sur Cloud Run

## Vérifier le déploiement

```
# Voir les services Cloud Run
gcloud run services list --region=europe-west1

# Obtenir les URLs
gcloud run services describe julia-geostat --region=europe-west1 --format='value(status.url)'
gcloud run services describe python-viz --region=europe-west1 --format='value(status.url)'
```

## Étape 4 : Connecter vos plateformes

### GEO-ECONOMIX (sur Abacus AI)

Dans le fichier `.env` de GEO-ECONOMIX, ajoutez :

```
JULIA_MICROSERVICE_URL=https://julia-geostat-XXXXX-ew.a.run.app
PYVISTA_MICROSERVICE_URL=https://python-viz-XXXXX-ew.a.run.app
MICROSERVICE_API_KEY=sk-solaris-votre-cle-geoekonomix
```

Puis activez les feature flags dans `lib/microservices/features.ts` :

```
export const MICROSERVICE_FEATURES = {
  JULIA_VARIOGRAPHY: true,    // ← activer
  JULIA_KRIGING: true,       // ← activer
  JULIA_SGS: true,           // ← activer
  JULIA_MONTE_CARLO: true,   // ← activer
  JULIA_PIT_OPTIMIZATION: true, // ← activer
  JULIA_BLOCK_MODEL: true,   // ← activer
  PYVISTA_3D_VIZ: true,      // ← activer
};
```

### GeoMatrix / TerraExploration

Même configuration, avec leur propre `MICROSERVICE_API_KEY`.

## Étape 5 : Monitoring

### Logs

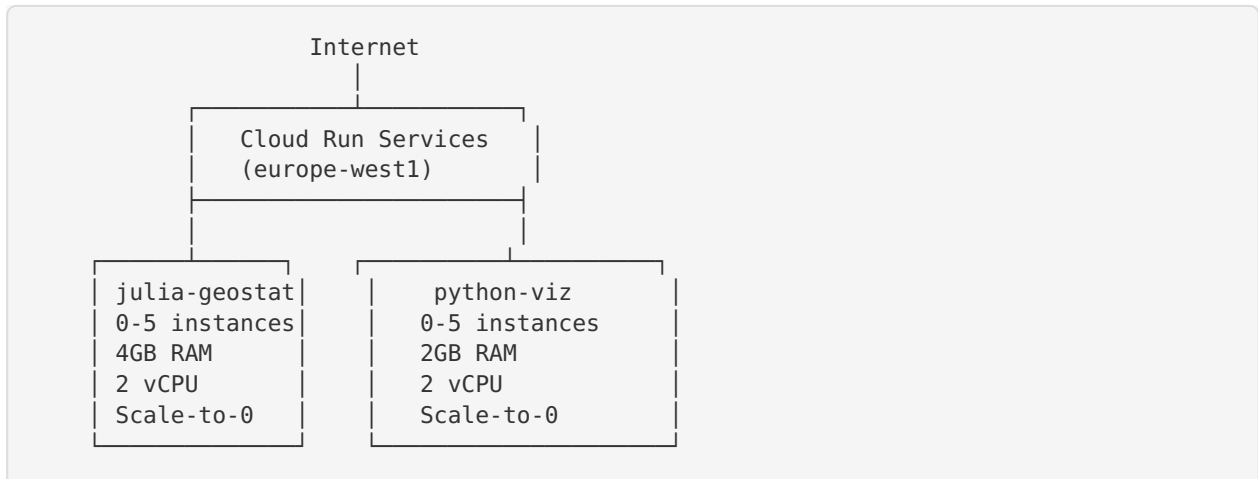
```
# Julia logs
gcloud run services logs read julia-geostat --region=europe-west1 --limit=50

# Python logs
gcloud run services logs read python-viz --region=europe-west1 --limit=50
```

### Métriques

Accédez au dashboard Cloud Run : <https://console.cloud.google.com/run>

## Architecture de production



### Domaine personnalisé (optionnel)

Pour utiliser `api.solarius-technology.com` :

```
gcloud run domain-mappings create \
  --service=julia-geostat \
  --domain=julia-api.solarius-technology.com \
  --region=europe-west1
```

## Dépannage

| Problème                | Solution                                                                                                                             |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Julia lent au 1er appel | Normal : JIT compilation (~30s). Les appels suivants sont rapides. Cloud Run garde l'instance chaude si <code>min-instances=1</code> |
| PyVista erreur de rendu | Vérifier que <code>PYVISTA_OFF_SCREEN=true</code> est défini                                                                         |
| 403 Forbidden           | Vérifier le header <code>X-API-Key</code>                                                                                            |
| Timeout 504             | Augmenter <code>--timeout</code> dans Cloud Run (max 3600s)                                                                          |
| Out of memory           | Augmenter <code>--memory</code> (max 32Gi)                                                                                           |